AIR UNIVERSITY ISLAMABAD

Kamra Campus

FULL STACK DEVELOPMENT LAB

Lab 07 Report

University API Access Management System

API key generation, role-based access control and request limiting using Node.js, Express.js, MySQL and jQuery AJAX

Student Name	Wasif Ali
Registration ID	245187
Class	BSCS-F-24-B (Semester BSCS VB)
Course	Full Stack Development (CS-301L)
Instructor	M. Fahad Qasim
Discipline	Computer Science
Date	October 22, 2026

Table of Contents

1. Introduction 3
1.1 Objectives 3
1.2 Technologies Used 3
2. System Architecture 3
2.1 Request Lifecycle 4
3. Database Design 4
3.1 Table Descriptions 4
3.2 Sample Data 5
4. Project Structure 5
5. Setup and Execution 5
6. API Endpoints 6
6.1 Administrative Endpoints 6
6.2 Student, Teacher and Shared Endpoints 6
7. Middleware and Security 6
7.1 Admin Authentication 6
7.2 User Authentication and Request Limiting 7
7.3 Preventing Access to Another Student's Records 7
7.4 Other Safeguards 7
7.5 HTTP Status Codes Used 8
8. Application Demonstration 8
8.1 Administrator Panel 8
8.2 Settings Page 12
8.3 Student Dashboard 12
8.4 Teacher Dashboard 13
9. Testing and Results 14
10. Requirements Compliance 16
11. Conclusion 16
11.1 Possible Improvements 16

1. Introduction

This report documents the design, implementation and testing of the University API Access Management System developed for Lab 07. The system allows an administrator to issue and manage API keys, lets students read only their own academic records, and lets teachers read the marks of every student. All access is controlled by API keys that are sent in the x-api-key HTTP header and validated by server-side middleware.

1.1 Objectives

- Design a relational MySQL database for students, teachers, courses, marks, API keys and request logs.
- Build an administrator panel at /admin for generating, limiting, monitoring and revoking API keys.
- Build a settings page where students and teachers save their assigned API key.
- Protect admin, student and teacher endpoints with separate middleware and role checks.
- Prevent a student from reading another student's records by changing the URL or query parameters.
- Store request usage in the database and enforce request limits with correct HTTP status codes.

1.2 Technologies Used

Technology	Purpose
Node.js	JavaScript runtime that executes the backend server.
Express.js	Web framework used for routing, middleware and serving the frontend pages.
MySQL (XAMPP)	Relational database storing all records, API keys and usage logs.
mysql package	Connection pool and parameterized (placeholder) queries from Node.js.
cors	Controls which browser origins and headers may call the API.
HTML, CSS	Responsive user interface for the admin, settings, student and teacher pages.
jQuery AJAX	Sends requests with the API key header and updates pages without reloading.

2. System Architecture

The browser never talks to the database directly. Every action is an AJAX request to the Express server. The server first runs the authentication middleware, which validates the API key, the role, the key status and the remaining request quota. Only then does the route handler run its SQL query and return JSON together with an HTTP status code.



Figure 1: High-level architecture and request flow

2.1 Request Lifecycle

1. jQuery sends a request such as GET /api/student/marks with the header x-api-key.
2. Express matches the route and runs the middleware attached to it.
3. **401** is returned if the key is missing or unknown.
4. **403** is returned if the key is revoked or belongs to a role that may not use the route.
5. **429** is returned if the key has no requests left. Otherwise one request is consumed atomically.
6. The route handler runs a parameterized SQL query and returns JSON that includes the usage counters.
7. The request and its status code are written to the api_logs table.

3. Database Design

The database university_api contains six InnoDB tables connected by foreign keys. Marks are stored as three components (assignment out of 20, midterm out of 30 and final exam out of 50) and the total out of 100 is calculated in SQL, so the data cannot become inconsistent.

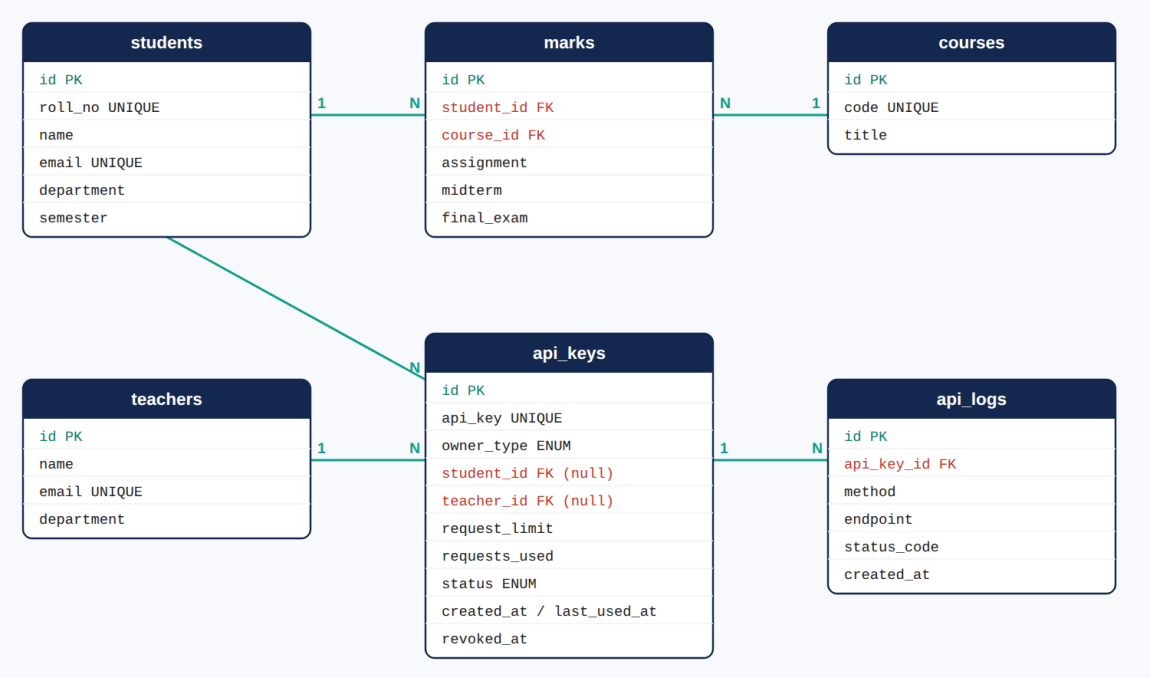


Figure 2: Entity relationship diagram of the university_api database

3.1 Table Descriptions

Table	Description
students	Student profile: roll number, name, email, department and semester.
teachers	Teacher profile: name, email and department.
courses	Course code and title.
marks	One row per student and course (unique pair) with assignment, midterm and final exam marks. Foreign keys to students and courses.
api_keys	Issued keys with owner type, owner (student_id or teacher_id), request_limit, requests_used, status (active or revoked) and timestamps.
api_logs	Usage history: one row per request made with a known key, with method, endpoint, status code and time.

Design decisions. An API key belongs to exactly one student or one teacher through two nullable foreign keys, so referential integrity is enforced by the database. Foreign keys use ON DELETE CASCADE, so removing a person also removes their keys and logs. The key value is UNIQUE and indexed for fast lookup on every request.

3.2 Sample Data

The SQL script database/university_api.sql creates the schema and inserts 6 students, 2 teachers, 4 courses and 24 marks records (every student has marks in every course), so the system can be demonstrated immediately after importing the file in phpMyAdmin.

4. Project Structure

```
university-api/
  server.js              Express app, static pages, 404 and error handler
  config.js              Port, admin API key, database settings
  db.js                  MySQL connection pool and promise-based query()
  middleware/auth.js     adminAuth and authenticate() middleware
  routes/admin.js        Administrative endpoints  (/api/admin/*)
  routes/api.js          Student and teacher endpoints  (/api/*)
  database/university_api.sql   Schema and sample data
  public/                Frontend (HTML, CSS, jQuery AJAX)
    admin.html, settings.html, student.html, teacher.html, index.html
    css/style.css
    js/common.js, admin.js, settings.js, student.js, teacher.js
```

5. Setup and Execution

1. Open the XAMPP Control Panel and start MySQL.
2. Open phpMyAdmin, choose Import and select database/university_api.sql to create the database and sample data.
3. Open the project folder in VS Code and open a terminal in that folder.
4. Run `npm install` and then `node server.js`.
5. Open `http://localhost:3000/admin` in the browser.

The terminal should report that the API is ready on port 3000 and that MySQL is connected, as shown below.

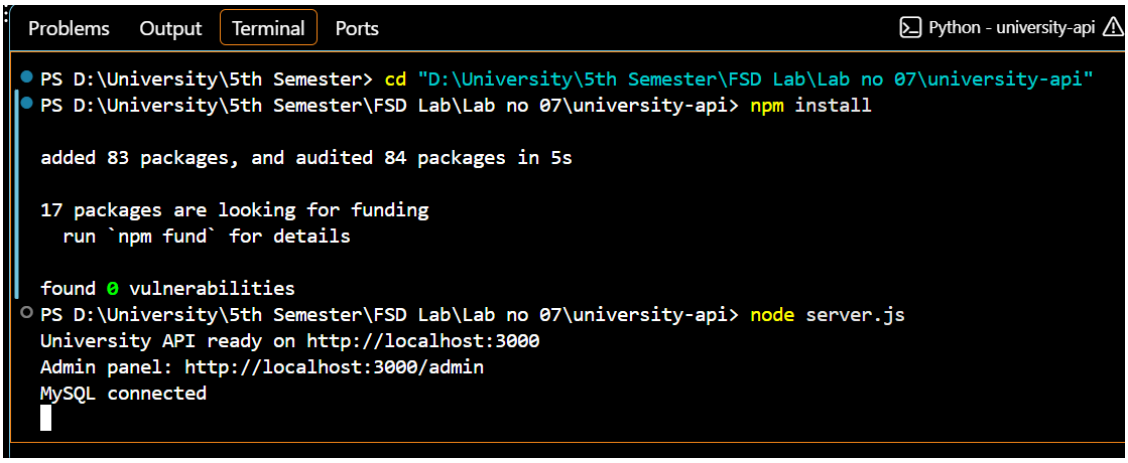


Figure 3: Server started successfully and connected to MySQL

Note. The frontend is served by the Node.js server itself. The pages must therefore be opened through http://localhost:3000 and not through an editor extension such as Live Server, which cannot run the API.

6. API Endpoints

Every endpoint expects the API key in the x-api-key request header. The admin endpoints require the predefined admin key defined in config.js. The other endpoints require a key issued to a student or a teacher.

6.1 Administrative Endpoints

Method	Endpoint	Purpose
GET	/api/admin/students	List all students
GET	/api/admin/teachers	List all teachers
GET	/api/admin/keys	List all keys with owner, usage and status
POST	/api/admin/keys	Generate a key (owner_type, owner_id, request_limit). Returns 201
PUT	/api/admin/keys/:id/limit	Update the request limit of a key
PUT	/api/admin/keys/:id/revoke	Revoke a key permanently
PUT	/api/admin/keys/:id/reset	Reset the usage counter to zero
GET	/api/admin/keys/:id/logs	Show the latest requests made with a key

6.2 Student, Teacher and Shared Endpoints

Method	Endpoint	Allowed role	Purpose
GET	/api/whoami	Student, Teacher	Verify a key and return the profile (does not use a request)
GET	/api/student/me	Student	Own profile
GET	/api/student/marks	Student	Own marks only
GET	/api/teacher/me	Teacher	Own profile
GET	/api/teacher/students	Teacher	All students
GET	/api/teacher/marks	Teacher	Marks of all students
GET	/api/students/:id/marks	Student, Teacher	Teacher: any student. Student: own id only, otherwise 403

7. Middleware and Security

7.1 Admin Authentication

The adminAuth middleware protects every route under /api/admin. A missing or unknown key returns 401. A valid student or teacher key used on an admin route returns 403, because the user is identified but not permitted. The comparison with the admin key uses a constant-time function.

7.2 User Authentication and Request Limiting

The authenticate(roles) middleware performs five checks in order: header present, key exists, key active, role allowed on the route and quota available. The quota check is a single atomic SQL statement, so two simultaneous requests cannot exceed the limit.

middleware/auth.js: atomic quota enforcement

```
const result = await query(
  `UPDATE api_keys
      SET requests_used = requests_used + 1, last_used_at = NOW()
    WHERE id = ? AND status = 'active' AND requests_used < request_limit`, [k.id]);

if (result.affectedRows === 0) {
  // no row updated -> revoked or limit reached
  return res.status(429).json({ message: 'Too Many Requests: API request limit reached' });
}
```

7.3 Preventing Access to Another Student's Records

Student routes never accept an identifier from the client. The student id is read from the database record of the API key (req.apiKey.owner_id). The only route that contains an id in the URL compares it with the id of the key owner and returns 403 if they differ.

routes/api.js: ownership check

```
router.get('/students/:id/marks', anyUser, async (req, res, next) => {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id < 1) {
    return res.status(400).json({ message: 'Invalid student id' });
  }
  if (req.apiKey.owner_type === 'student' && id !== req.apiKey.owner_id) {
    return res.status(403).json({ message: 'Forbidden: you can only view your own records' });
  }
  ...
});
```

7.4 Other Safeguards

- All SQL statements use placeholders, so user input is never concatenated into SQL (protection against SQL injection).
- Input is validated: owner type, owner id and request limit return 400 when invalid; unknown owners return 404.
- Only one active key is allowed per person; generating a second one returns 409 Conflict.
- Frontend output is HTML-escaped to prevent cross-site scripting.
- A central error handler returns JSON errors (400 for malformed JSON, 404 for unknown endpoints, 500 for server errors).

7.5 HTTP Status Codes Used

Code	Meaning	When it is returned
200	OK	Data returned successfully
201	Created	API key generated
400	Bad Request	Invalid input or malformed JSON
401	Unauthorized	API key missing or invalid
403	Forbidden	Revoked key, wrong role, or student requesting another student's record
404	Not Found	Owner, key, student or endpoint does not exist
409	Conflict	The person already has an active key
429	Too Many Requests	Request limit of the key has been used up
500	Internal Server Error	Unexpected database or server error

8. Application Demonstration

8.1 Administrator Panel

The administrator logs in with the predefined admin API key. The key is kept only for the browser session and is sent in the x-api-key header with every admin request.

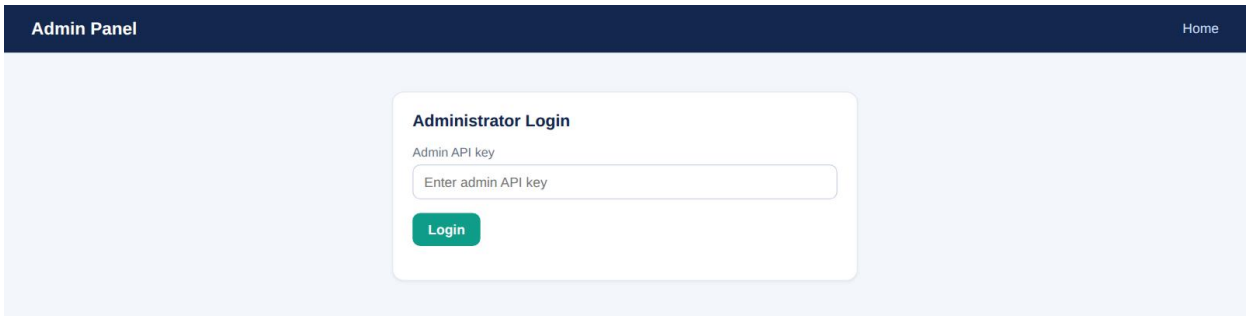


Figure 4: Administrator login

An API key is generated by selecting the owner type, the owner and the request limit. The new key is displayed once so that it can be copied and given to the user.

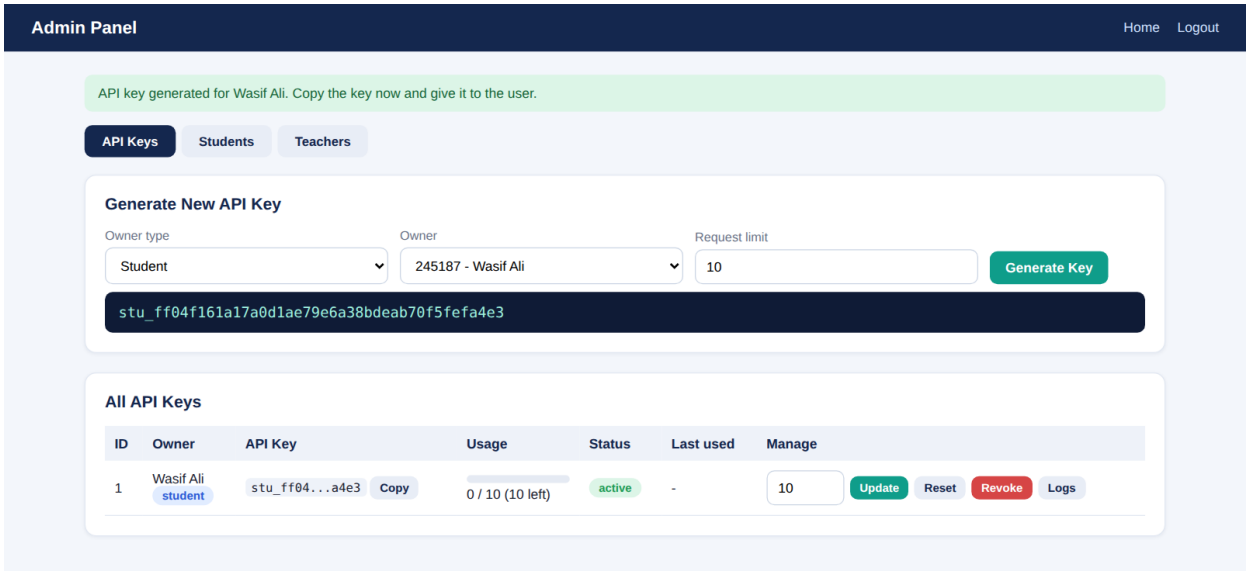


Figure 5: Generating an API key for a student (HTTP 201)

The API Keys table shows the owner, a masked key with a copy button, a usage bar, the status, the last time the key was used and management actions.

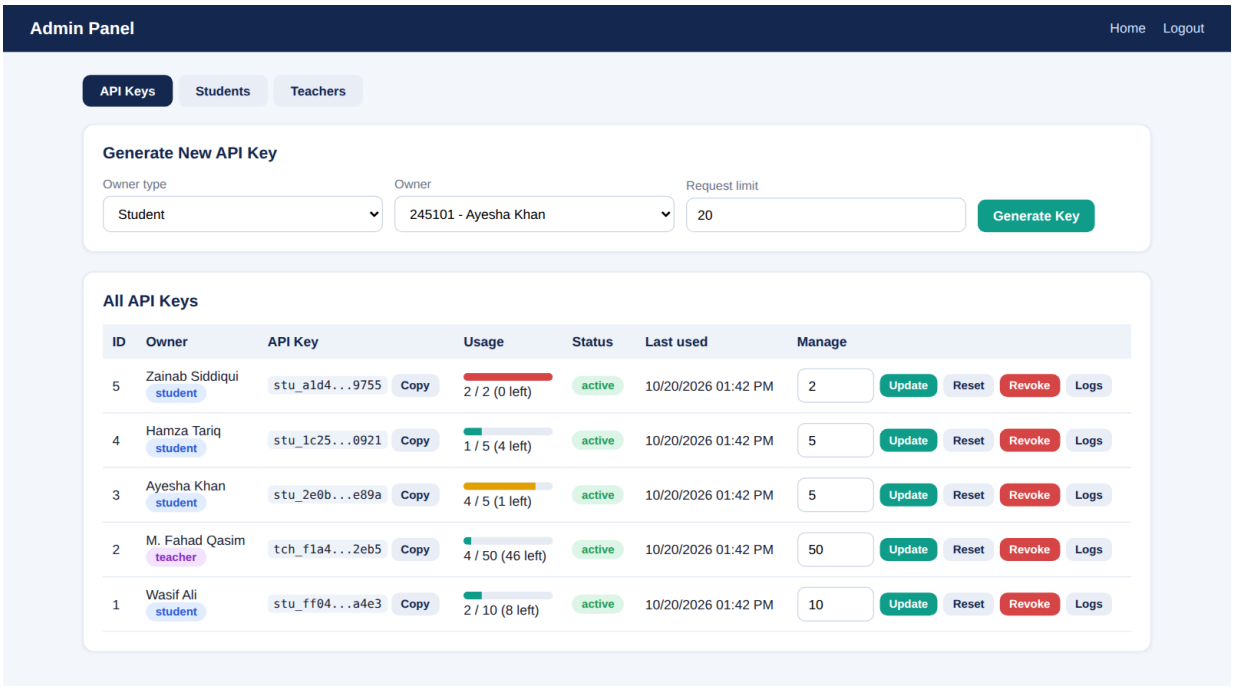


Figure 6: API keys with usage, status and management actions

The request limit can be changed at any time. In the example below the limit of one student was updated from 5 to 8.

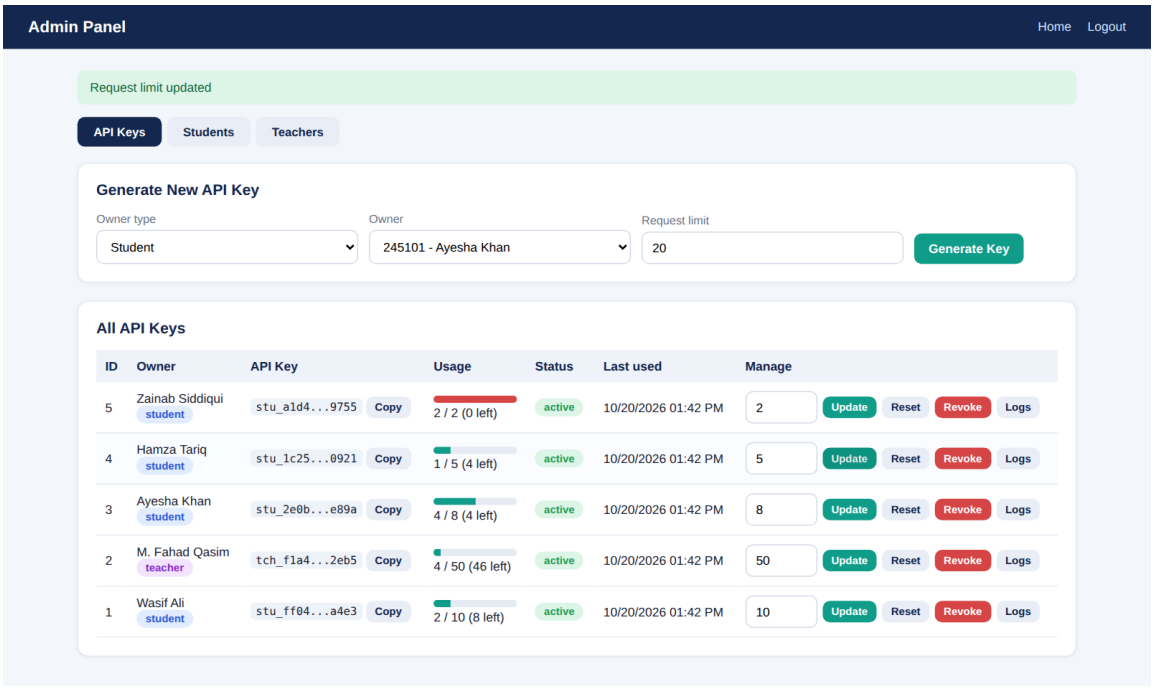


Figure 7: Updating the request limit of a key

The Logs button shows the most recent requests made with a key, including the endpoint and the HTTP status code. Rejected requests, such as the 403 caused by calling a teacher endpoint with a student key, are logged as well.

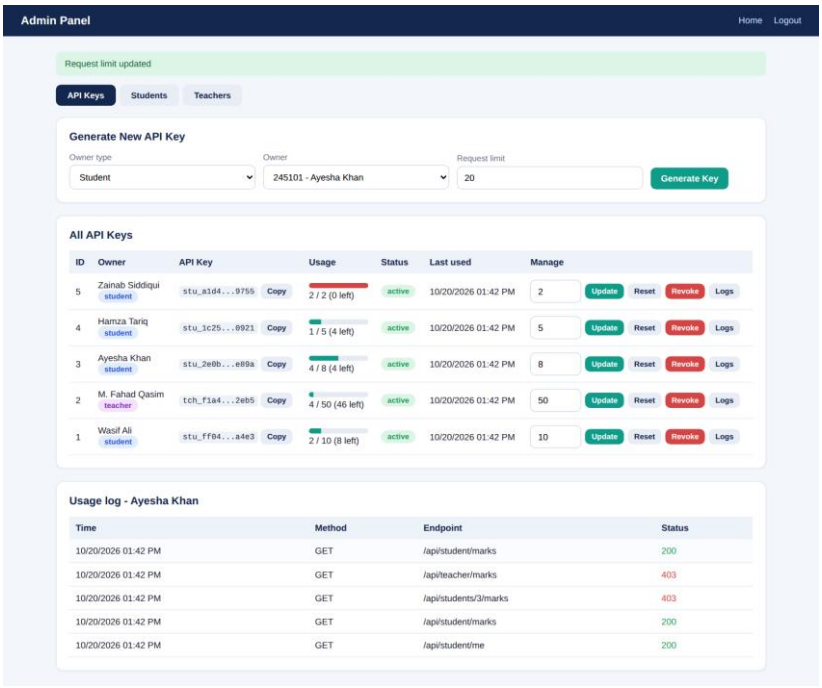


Figure 8: Usage history of an API key

A key can be revoked after confirmation. A revoked key is marked in red and can no longer be used.

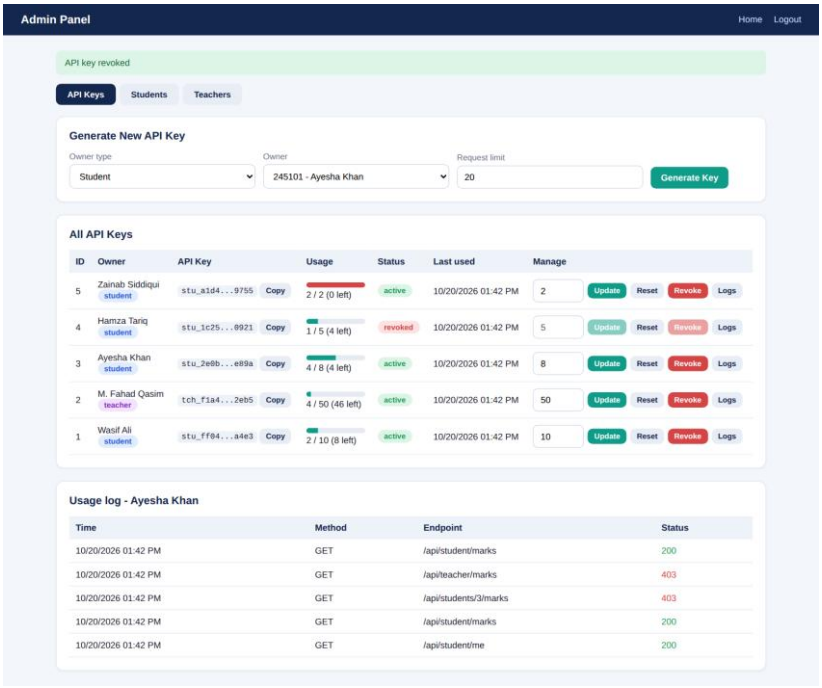


Figure 9: Revoked key shown in the keys table

The Students and Teachers tabs let the administrator view all registered users.

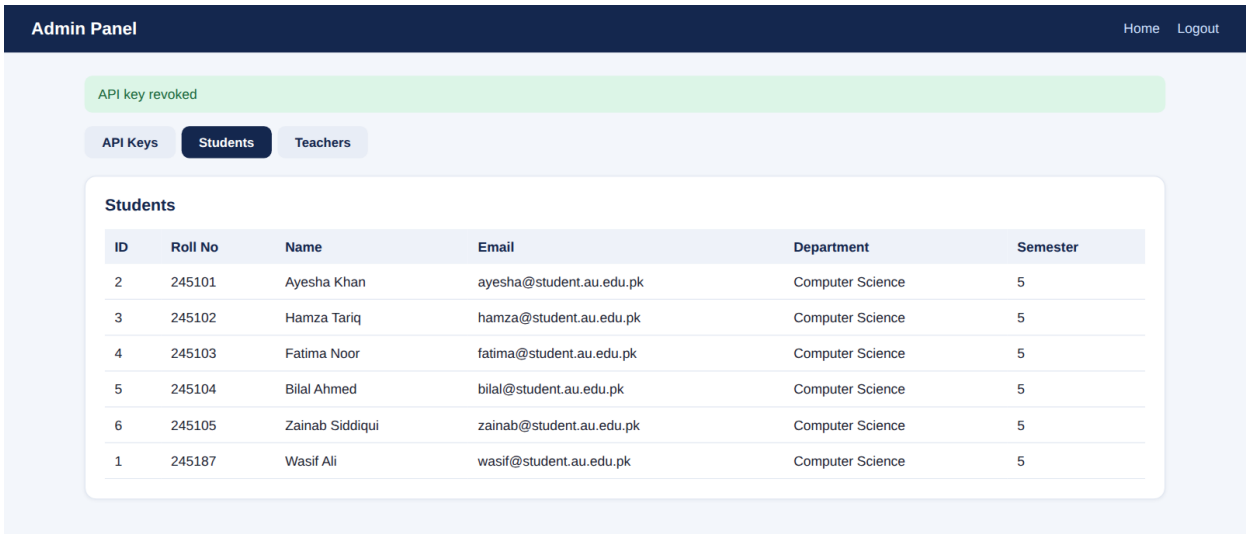


Figure 10: Students tab

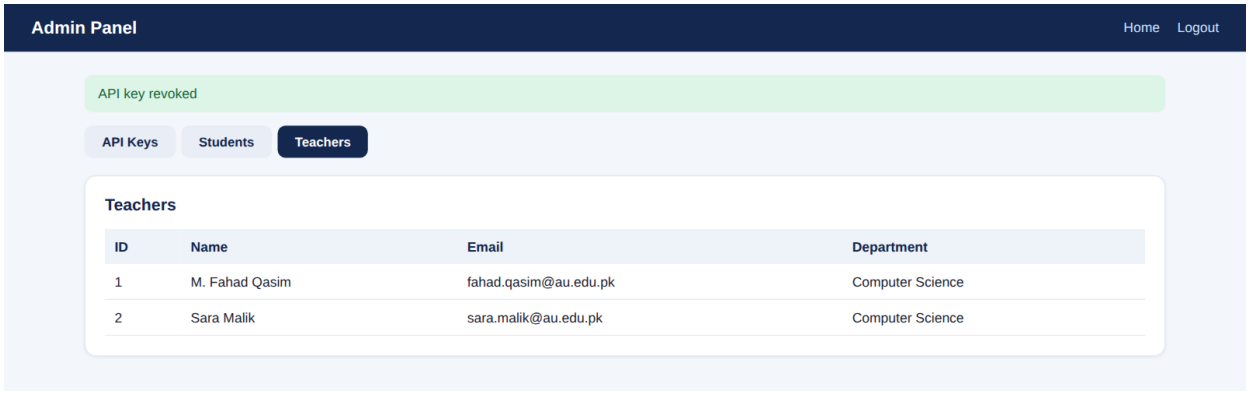


Figure 11: Teachers tab

8.2 Settings Page

Students and teachers enter their key on the settings page. The key is verified with the server before it is saved, so an invalid key is rejected immediately. Verification uses the /api/whoami endpoint, which does not consume a request.

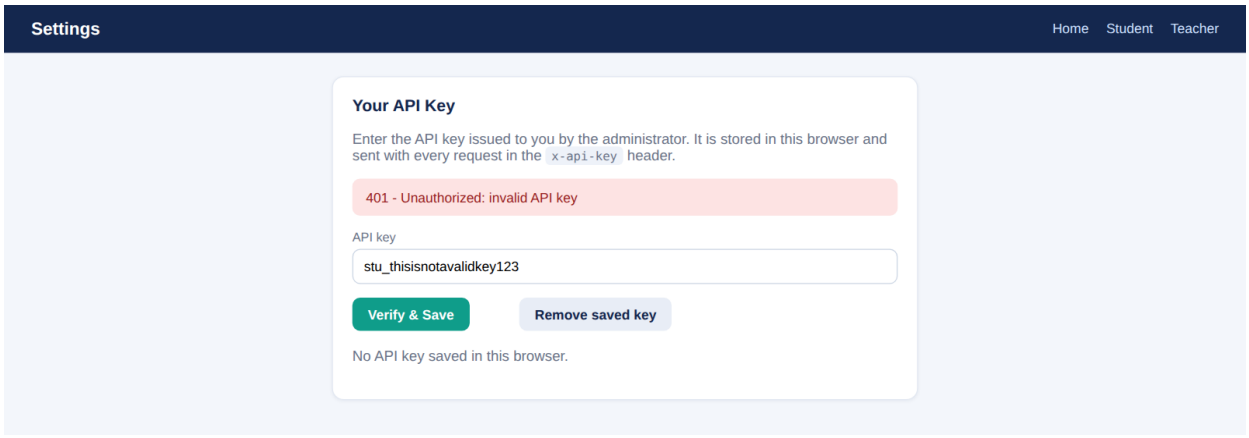


Figure 12: Invalid key rejected with HTTP 401

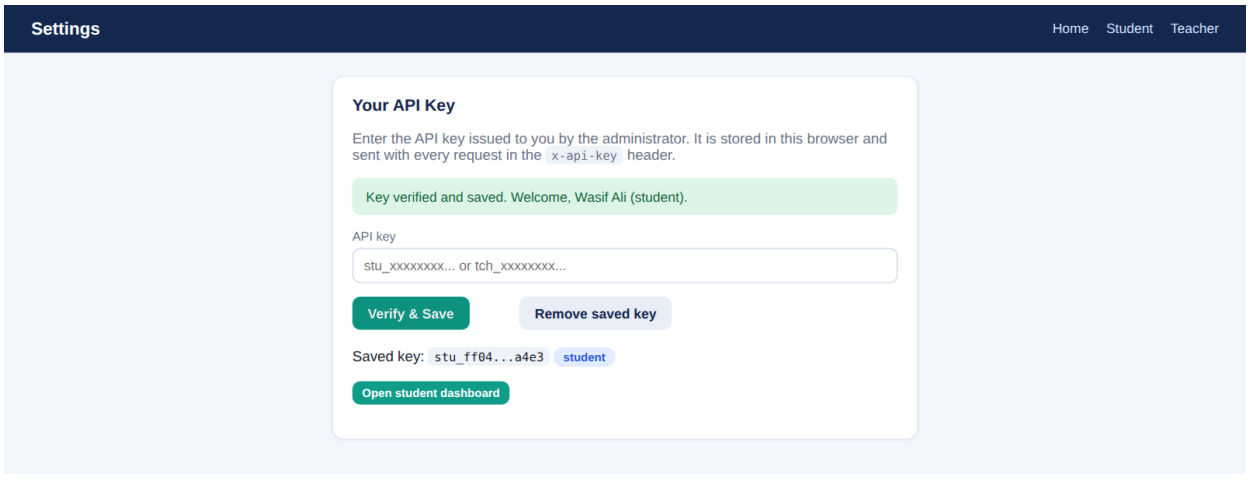


Figure 13: Valid key verified and saved

8.3 Student Dashboard

The student dashboard shows only the profile and marks that belong to the owner of the saved key, together with the remaining request quota. A pass or fail result is calculated from the total marks.

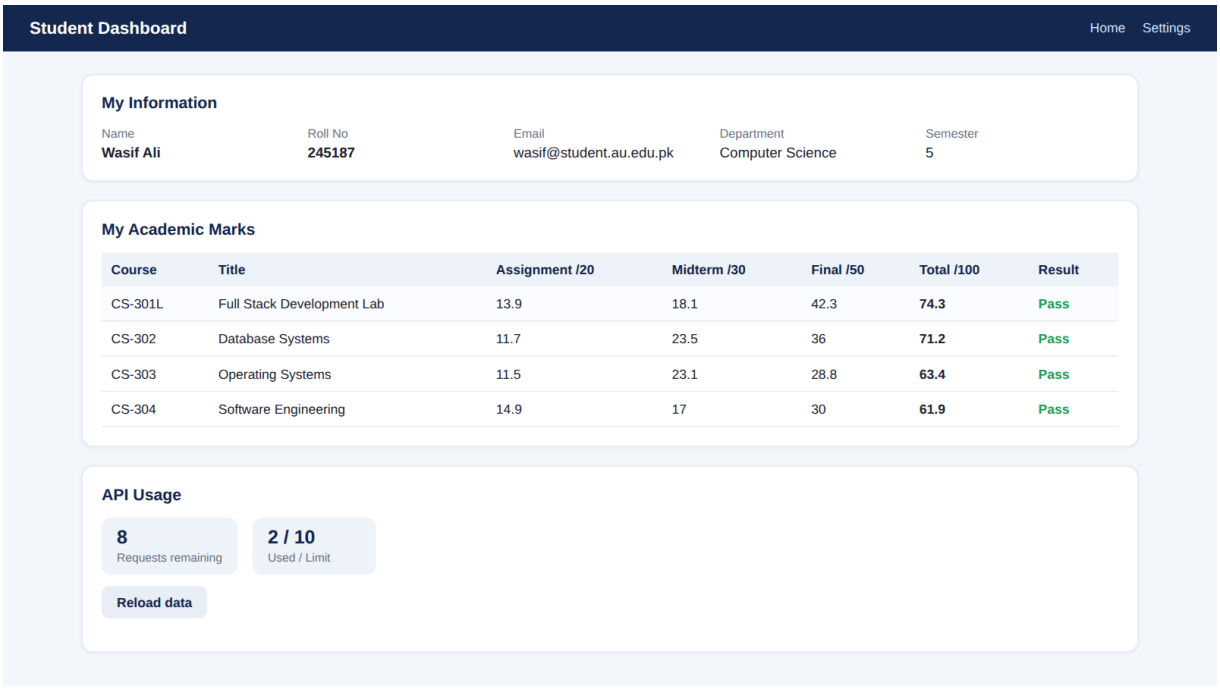


Figure 14: Student dashboard showing own information and marks

8.4 Teacher Dashboard

The teacher dashboard shows the marks of all students. The table can be filtered without extra API requests, and the teacher can also load a single student through the /api/students/:id/marks endpoint.

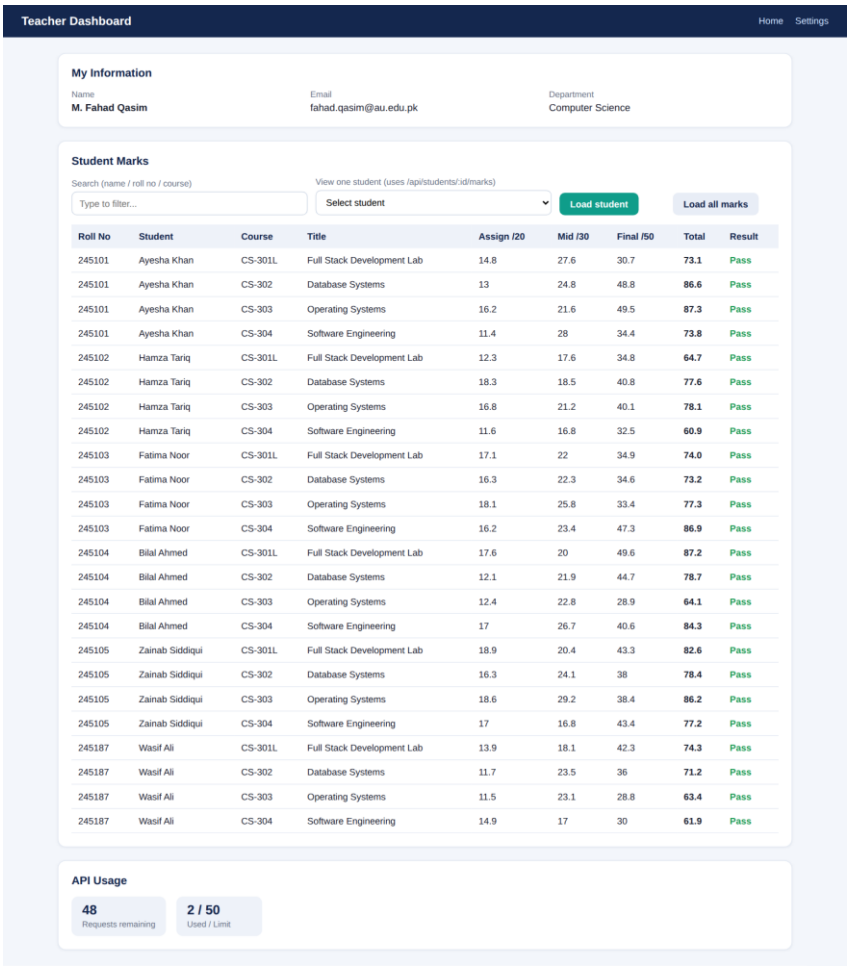


Figure 15: Teacher dashboard showing marks of all students

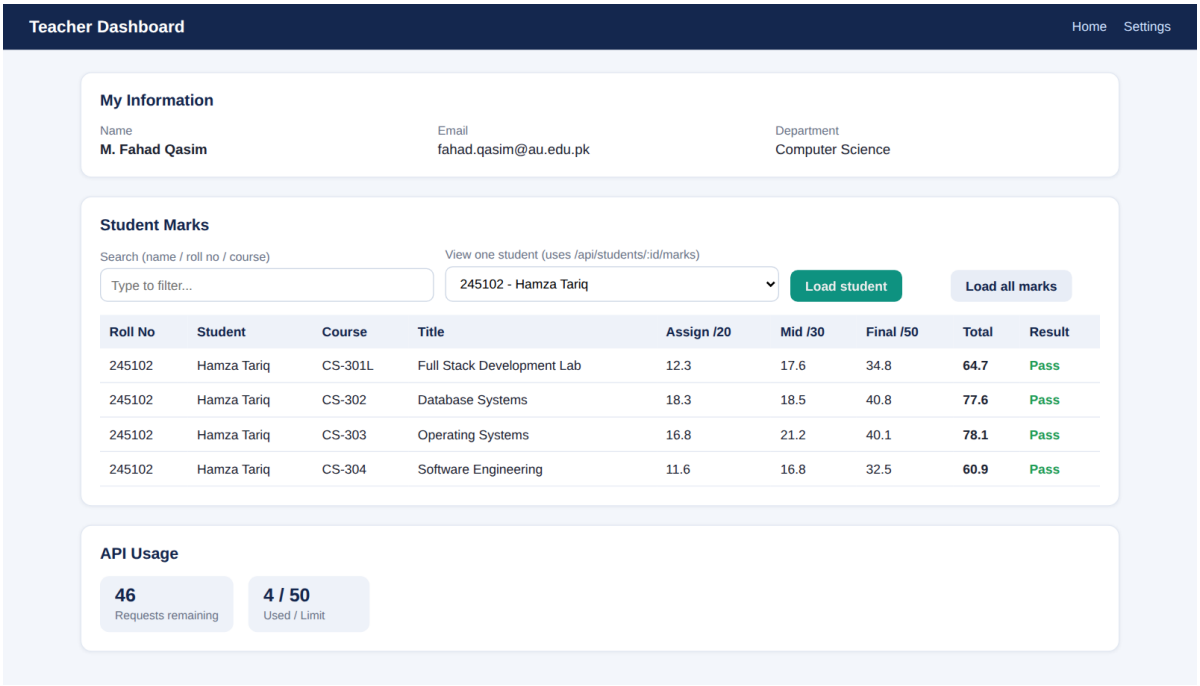


Figure 16: Teacher viewing the marks of one selected student

9. Testing and Results

The following tests were executed against the running application. Every test produced the expected HTTP status code and message.

No.	Test scenario	Expected behaviour	Status	Result
T1	Admin endpoint without a key	Request rejected	401	Pass
T2	Admin endpoint with an invalid key	Request rejected	401	Pass
T3	Student key used on an admin endpoint	Access denied	403	Pass
T4	Admin generates a key for a student	Key created	201	Pass
T5	Second key for the same person	Conflict reported	409	Pass
T6	Invalid request limit or malformed JSON	Validation error	400	Pass
T7	Student reads own marks	Own data returned	200	Pass
T8	Student changes the id in the URL	Access denied	403	Pass
T9	Student calls a teacher endpoint	Access denied	403	Pass
T10	Teacher reads marks of all students	All data returned	200	Pass
T11	Student exceeds the request limit	Request blocked	429	Pass
T12	Revoked key is used	Access denied	403	Pass
T13	Usage counter and logs after requests	Stored in database	–	Pass

The access-control tests were repeated from the command line. The output below shows the missing key, invalid key, own record, changed id, wrong role and wrong endpoint cases.

```
# Missing key
$ curl http://localhost:3000/api/student/me
HTTP 401 Unauthorized
{
 "message": "Unauthorized: x-api-key header is missing"
}

# Invalid key
$ curl http://localhost:3000/api/student/me -H "x-api-key: stu_wron..."
HTTP 401 Unauthorized
{
 "message": "Unauthorized: invalid API key"
}

# Student reads own record (own id = 1)
$ curl http://localhost:3000/api/students/1/marks -H "x-api-key: stu_ff04..."
HTTP 200 OK
{ "student": { "id": 1, "roll_no": "245187", "name": "Wasif Ali", ... },
  "data": [ ...4 course records... ],
  "usage": {"limit": 10, "used": 5, "remaining": 5} }

# Student changes the id in the URL (id = 2)
$ curl http://localhost:3000/api/students/2/marks -H "x-api-key: stu_ff04..."
HTTP 403 Forbidden
{
 "message": "Forbidden: you can only view your own records"
}

# Student calls a teacher endpoint
$ curl http://localhost:3000/api/teacher/marks -H "x-api-key: stu_ff04..."
HTTP 403 Forbidden
{
 "message": "Forbidden: student keys cannot access this endpoint"
}

# Student key used on an admin endpoint
$ curl http://localhost:3000/api/admin/keys -H "x-api-key: stu_ff04..."
HTTP 403 Forbidden
{
 "message": "Forbidden: admin API key required"
}
```

Figure 17: Access-control tests executed with curl

The two screenshots below show the dashboard behaviour when a key has no requests left and when a key has been revoked.

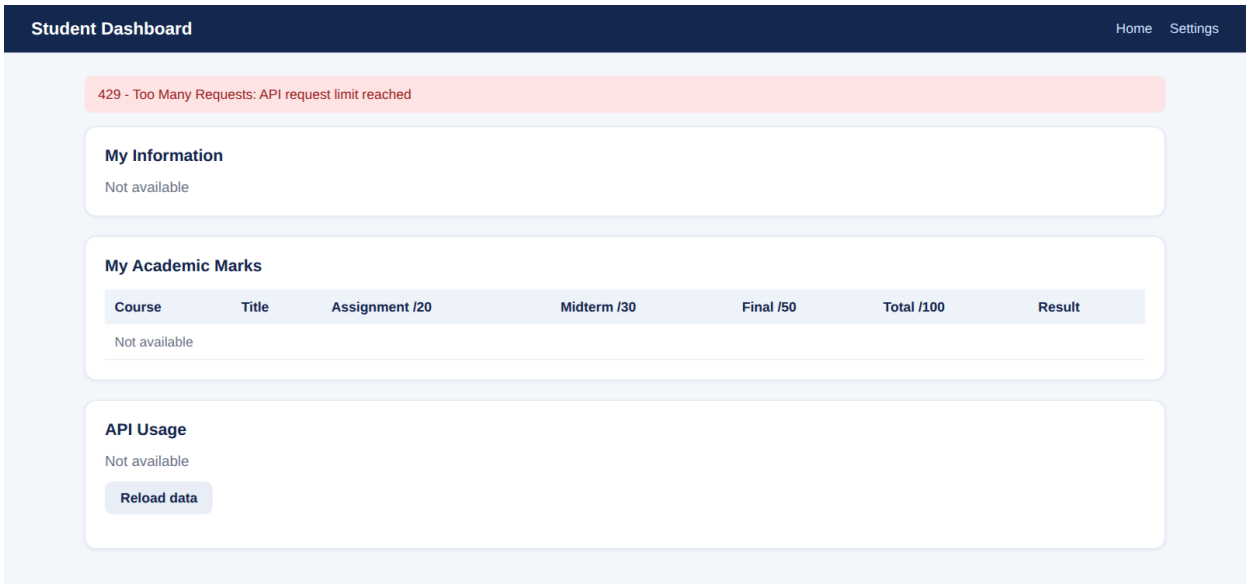


Figure 18: Request limit reached (HTTP 429)

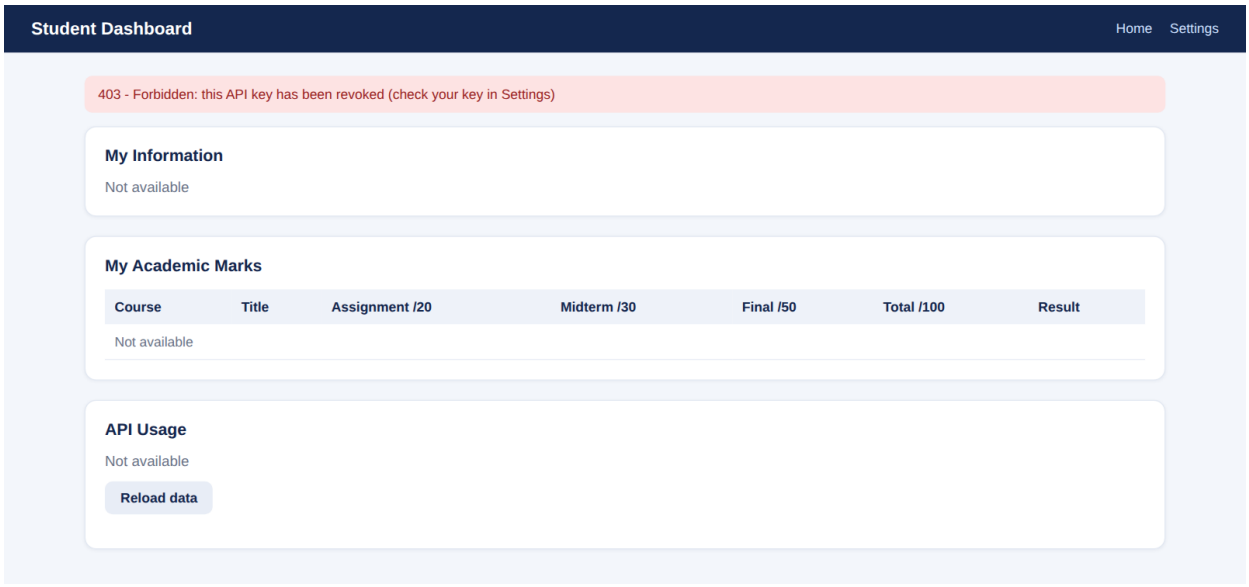


Figure 19: Revoked key rejected (HTTP 403)

10. Requirements Compliance

#	Requirement	Implementation
1	Database for students, teachers, marks and API keys	Six related tables with sample data (Section 3)
2	Admin panel at /admin	View users, generate keys, set and update limits, view usage and status, revoke keys (Section 8.1)
3	Settings page for saving keys	/settings verifies and stores the key (Section 8.2)
4	Students see own data; teachers see all	Student and teacher endpoints and dashboards (Sections 8.3 and 8.4)
5	Admin key and user keys sent in the header	x-api-key header validated by adminAuth and authenticate (Section 7)
6	Suitable endpoints and HTTP methods	GET, POST and PUT endpoints (Section 6)
7	Middleware and no cross-student access	Role checks and ownership check on the id (Section 7.3)
8	Usage in database, limits and error handling	requests_used and api_logs tables, 429 limit, status codes (Sections 7.2 and 7.5)

11. Conclusion

The University API Access Management System meets all requirements of Lab 07. Access is granted only through API keys that the administrator can generate, limit, monitor and revoke. Students are restricted to their own records, teachers can view all marks, usage is stored in the database, and every failure is reported with an appropriate HTTP status code. The lab also demonstrates the main ideas of server-side authorization: validating credentials on every request, separating roles, using atomic database updates for quotas, and never trusting identifiers supplied by the client.

11.1 Possible Improvements

- Store only hashed API keys in the database and show the full key once at creation.
- Replace the predefined admin key with administrator accounts and sessions.

- Add time-based limits (requests per day or hour) and serve the application over HTTPS in production.